

Git & GitHub Workshop

Complete Guide for Beginners

📅 2 Hours Session — 👥 Batch 25 Freshers

Workshop Overview

🕒 **Duration:** 2 Hours

💻 **Format:** Hands-on + Theory

🎓 **Audience:** Batch 25 Freshers (Beginners)

🔧 **Prerequisites:** Git installed, GitHub account

👤 **Prepared By**

Muhammad Idrees

Head of Workshops

FAST Data Science Society

Contents

1	Pre-Workshop Setup	2
2	Detailed Workshop Schedule	2
2.1	Session 1: Introduction & Git Basics (0:00 - 0:45)	2
2.1.1	Part A: Why Git & GitHub? (10 minutes)	2
2.1.2	Part B: Git Fundamentals (20 minutes)	3
2.1.3	Part C: Hands-on Practice (15 minutes)	4
2.2	Session 2: GitHub & Collaboration (0:50 - 1:30)	5
2.2.1	Part A: Connecting to GitHub (15 minutes)	5
2.2.2	Part B: Cloning & Collaboration Basics (15 minutes)	6
2.2.3	Part C: Branching Basics (10 minutes)	6
2.3	Session 3: Practical Application (1:30 - 2:00)	7
2.3.1	Part A: Hands-on Group Exercise (15 minutes)	7
2.3.2	Part B: Common Issues & Solutions (10 minutes)	8
2.3.3	Part C: Best Practices & Next Steps (5 minutes)	9
3	Git Command Cheat Sheet	10
4	Materials Needed	11
4.1	For Organizers	11
4.2	For Participants	11
5	Tips for Conducting the Workshop	12
6	Post-Workshop Resources	13
7	Assessment & Feedback	13
7.1	Optional Quiz Questions	13
7.2	Feedback Collection	14
8	Troubleshooting Guide	15

1 Pre-Workshop Setup

⚠ Important: Complete Before Workshop

Send this checklist to all participants **1 day before** the workshop to ensure smooth execution.

✓ Checklist for Participants

1. Install Git

- Windows/Mac/Linux: <https://git-scm.com/downloads>
- Verify installation: Open terminal and type `git --version`

2. Create GitHub Account

- Register at: <https://github.com>
- Choose a professional username (will be visible to employers!)
- Verify email address

3. Install Code Editor

- Recommended: VS Code (<https://code.visualstudio.com>)
- Alternatives: Sublime Text, Atom, Notepad++

4. Basic Terminal Familiarity (Optional)

- Know how to navigate directories (`cd`, `ls/dir`)
- Create folders and files

💡 Pro Tip

Have participants bring **chargers** and ensure **stable WiFi** is available for GitHub operations.

2 Detailed Workshop Schedule

2.1 Session 1: Introduction & Git Basics (0:00 - 0:45)

2.1.1 Part A: Why Git & GitHub? (10 minutes)

? What is Version Control?

- **The Problem:** Tracking changes, collaboration chaos, lost work
- **Real Scenario:** Show example of "project_final.v2_final_actual.docx" naming mess
- **Solution:** Version control tracks every change systematically

Git vs GitHub - Understanding the Difference

</> Git

Version control system
Works locally on your computer
Command-line tool
Free and open-source

🐙 GitHub

Cloud hosting platform
Online service for collaboration
Web-based interface
Owned by Microsoft

💡 Analogy:

- **Git** = Word's "Track Changes" feature
- **GitHub** = Google Drive for code (but better!)

Industry Relevance

- Every tech company uses Git
- Essential for internships
- Required for placements
- Open source contribution
- Portfolio building
- Team collaboration skill

2.1.2 Part B: Git Fundamentals (20 minutes)

Core Concepts Explained

Repository

Your project folder with complete history of all changes

Commit

A snapshot of your project at a specific point in time (like a checkpoint in a game)

Branch

A parallel version of your code (work on features without breaking main code)

Three Stages

1. **Working Directory:** Where you edit files
2. **Staging Area:** Files ready to be committed
3. **Repository:** Permanent history of commits



Live Demo: First Git Commands

```
# Configure Git with your identity
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Initialize a new repository
git init

# Check the status of your repository
git status

# Stage all files for commit
git add .

# Commit the staged changes
git commit -m "Initial commit"

# View commit history
git log
```

i Command Explanations

<code>init</code>	Initialize a new Git repository in current folder
<code>status</code>	Check what files have changed
<code>add</code>	Stage files for commit (prepare them)
<code>commit</code>	Save a snapshot with a message
<code>log</code>	View history of all commits

2.1.3 Part C: Hands-on Practice (15 minutes)

✓ Exercise 1: Create Your First Repository

Students will practice the complete Git workflow from initialization to commit.

☰ Step-by-Step Instructions:

1. Create Project Folder

```
mkdir my-first-repo
cd my-first-repo
```

2. Create a Simple File

- Create `index.html` or `hello.txt`
- Add some content (e.g., "Hello Git!")

3. Initialize Git

```
git init
git status # Should show untracked files
```

4. Make First Commit

```
git add .
git status # Files should be staged (green)
git commit -m "Initial commit: Add hello file"
```

5. Modify File and Commit Again

- Edit the file, add more content
- Check status, stage, and commit

```
git status
git add .
git commit -m "Update: Add more content"
```

6. View Commit History

```
git log
git log --oneline # Compact view
```

⚠ Common Mistakes to Watch For

- **Forgetting to stage:** Remind students to use `git add` before commit
- **Empty commit messages:** Explain importance of descriptive messages
- **Wrong directory:** Make sure they're in the project folder
- **Not configured:** Check if name/email is set

☕ Break Time

0:45 - 0:50 (5 minutes)
Quick stretch break

2.2 Session 2: GitHub & Collaboration (0:50 - 1:30)

2.2.1 Part A: Connecting to GitHub (15 minutes)

🔗 Creating Remote Repository

1. Live Demo: Create Repository on GitHub

- Log into GitHub
- Click "New Repository" button
- Choose repository name (match local folder name)
- Add description (optional but recommended)
- Choose visibility (Public/Private)

2. Public vs Private Repositories

- **Public:** Anyone can see (good for portfolio, open source)
- **Private:** Only you and collaborators can see

3. README.md Importance

- First thing visitors see
- Explains what your project does
- Include: description, usage, technologies

📦 Push Your Local Repo to GitHub

```
# Connect local repo to GitHub
git remote add origin <URL>

# Rename branch to main (if needed)
git branch -M main

# Push commits to GitHub
git push -u origin main
```

i Important Terms

remote	Connection to GitHub server (cloud storage)
origin	Default name for the remote repository
push	Send your local commits to GitHub
pull	Get commits from GitHub to your local machine
-u flag	Sets upstream, so future pushes are easier

2.2.2 Part B: Cloning & Collaboration Basics (15 minutes)**📁 Cloning Repositories**

```
# Clone an existing repository
git clone <repository-URL>

# This creates a folder and downloads all files
```

? When to Clone vs Init?

Use git init	Use git clone
Starting a brand new project	Working on existing project
No repository exists yet	Repository already on GitHub
You're creating from scratch	Joining a team project

👥 Basic Collaboration Workflow

```
# 1. Get latest changes from GitHub
git pull

# 2. Make your changes to files
# (edit files in your editor)

# 3. Stage and commit your changes
git add .
git commit -m "Describe what you changed"

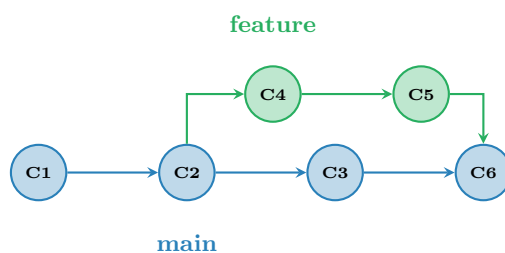
# 4. Send changes back to GitHub
git push
```

⚠️ Golden Rule of Collaboration

Always pull before you push! This ensures you have the latest changes and reduces conflicts.

2.2.3 Part C: Branching Basics (10 minutes)**💡 Why Branches?**

- Work on new features without breaking main code
- Multiple people can work simultaneously on different features
- Test ideas safely before merging
- Keep main/master branch stable and working



>_ Basic Branch Commands

```
# Create a new branch
git branch feature-name

# Switch to that branch
git checkout feature-name

# Create and switch in one command
git checkout -b feature-name

# List all branches (* shows current)
git branch

# Switch back to main
git checkout main

# Merge feature branch into main
git merge feature-name

# Delete a branch (after merging)
git branch -d feature-name
```

💡 Branch Naming Convention

Use descriptive names:

- feature/user-authentication
- bugfix/login-error
- improvement/optimize-loading

2.3 Session 3: Practical Application (1:30 - 2:00)

2.3.1 Part A: Hands-on Group Exercise (15 minutes)

✓ Exercise 2: Collaborative Project Simulation

Students will experience real collaboration by working in pairs.

👥 Setup: Pairing Students

1. Divide class into pairs (Person A & Person B)
2. Ensure both have working Git and GitHub access

☰ Step-by-Step Collaboration Exercise:

1. Person A: Create Repository

- Create a new repo on GitHub (public)
- Initialize with README
- Add Person B as collaborator (Settings → Collaborators)

2. Person B: Accept Invitation

- Check email for invitation
- Accept collaboration invite

3. Both: Clone Repository

```
git clone <repository-URL>
cd <repository-name>
```

4. Person A: Create Branch & Add File

```
git checkout -b person-a-feature
echo "Hello from Person A" > person-a.txt
git add person-a.txt
git commit -m "Add Person A's file"
git push origin person-a-feature
```

5. Person B: Create Branch & Add File

```
git checkout -b person-b-feature
echo "Hello from Person B" > person-b.txt
git add person-b.txt
git commit -m "Add Person B's file"
git push origin person-b-feature
```

6. Both: View Branches on GitHub

- Go to repository on GitHub
- See both branches listed
- Introduction to pull requests (brief demo)

2.3.2 Part B: Common Issues & Solutions (10 minutes)

🔧 Fixing Mistakes

Wrong Message

```
git commit --amend -m "Corrected message"
```

↶ Unstage Files

```
git reset HEAD <filename>
```

Discard Changes

```
git checkout -- <filename> # Warning: Cannot be undone!
```

🔍 View Differences

```
git diff # Unstaged changes
git diff --staged # Staged changes
```

⚠ Merge Conflicts

What is a Merge Conflict?

A merge conflict occurs when:

- Two people edit the same line in a file
- Git doesn't know which change to keep
- You must manually resolve it

How Conflicts Look:

```
<<<<<<< HEAD
This is my version of the code
=====
This is their version of the code
>>>>>>> feature-branch
```

Resolution Steps:

1. Open the conflicting file
2. Choose which version to keep (or combine both)
3. Remove conflict markers (<<<<<<<, =====, >>>>>>>)
4. Stage and commit the resolved file

2.3.3 Part C: Best Practices & Next Steps (5 minutes)

✓ ✓ Git Best Practices

- | | |
|----------------------------------|------------------------------------|
| • Commit often, commit logically | • Use <code>.gitignore</code> file |
| • Write meaningful messages | • Keep commits focused |
| • Pull before you push | • Test before committing |
| • Don't commit sensitive data | • Use branches for features |

🔑 What's Next?

- **Resources:** GitHub Learning Lab, Interactive Git tutorials
- **Advanced Topics:** Pull requests, GitHub Actions, Git rebase
- **Portfolio:** Push all projects to GitHub, contribute to open source

3 Git Command Cheat Sheet

> Essential Git Commands Reference

🔧 Setup & Configuration

```
git config --global user.name "Your Name"
git config --global user.email "email@example.com"
git config --list # View all settings
```

▶ Starting a Repository

```
git init # Initialize new repository
git clone <url> # Copy existing repository
```

Basic Workflow

```
git status # Check status
git add <file> # Stage specific file
git add . # Stage all changes
git commit -m "message" # Save snapshot
git log # View history
git log --oneline # Compact view
```

🐙 Working with GitHub

```
git remote add origin <url> # Connect to GitHub
git push -u origin main # Upload to GitHub
git push # Push after first time
git pull # Download from GitHub
git fetch # Check for updates
```

🔗 Branching

```
git branch # List all branches
git branch <name> # Create new branch
git checkout <name> # Switch to branch
git checkout -b <name> # Create & switch
git merge <name> # Merge branch
git branch -d <name> # Delete branch
```

↶ Undoing Changes

```
git checkout -- <file> # Discard local changes
git reset HEAD <file> # Unstage file
git commit --amend # Modify last commit
git revert <commit-hash> # Undo specific commit
```

📄 Information Commands

```
git diff # View unstaged changes
git diff --staged # View staged changes
git show <commit-hash> # Show specific commit
git remote -v # View remote URLs
```

4 Materials Needed

4.1 For Organizers

Preparation Checklist

-  Projector/Screen for live demonstrations
-  Instructor laptop with Git and GitHub configured
-  Sample repository prepared beforehand
- Handout with command cheat sheet
-  Ensure stable WiFi access
-  2-3 Teaching Assistants for hands-on help
-  Whiteboard/markers for explanations

4.2 For Participants

Student Requirements

-  Laptop with Git installed
-  GitHub account created and verified
-  Code editor installed (VS Code recommended)
-  Internet connection
-  Laptop charger

5 Tips for Conducting the Workshop

Instructor Guidelines

1. Keep it Interactive

- Ask questions frequently
- Encourage student participation
- Use real-world analogies

2. Go Slow with Beginners

- Freshers need time to absorb concepts
- Pause after each command for questions
- Check understanding before moving forward

3. Use Real Examples

- Relate to group projects and assignments
- Show actual repository examples
- Demonstrate mistakes and how to fix them

4. Handle Errors Live

- Don't hide mistakes during demos
- Show debugging process
- Normalize making errors

5. Have Teaching Assistants

- 2-3 TAs for hands-on sessions
- Walk around during exercises
- Help with individual problems

6 Post-Workshop Resources

Learning Resources to Share

Online Tutorials

- **Official Git Documentation:** <https://git-scm.com/doc>
- **GitHub Learning Lab:** <https://lab.github.com/>
- **Interactive Git Tutorial:** <https://learngitbranching.js.org/>
- **GitHub Skills:** <https://github.com/skills>
- **Atlassian Git Tutorial:** <https://www.atlassian.com/git/tutorials>

Video Tutorials

- Git and GitHub for Beginners (freeCodeCamp)
- Git Tutorial for Beginners (Programming with Mosh)
- GitHub Ultimate Course (Traversy Media)

Practice Platforms

- Learn Git Branching (Interactive game)
- GitHub Skills exercises
- Try Git (Interactive tutorial)

7 Assessment & Feedback

7.1 Optional Quiz Questions

Quick Knowledge Check

1. What is the difference between Git and GitHub?
2. What command initializes a new Git repository?
3. What are the three stages in Git's architecture?
4. What does `git add` do?
5. Why should you write meaningful commit messages?
6. What is a branch in Git?
7. When should you use `git pull`?
8. How do you undo changes to a file before staging?
9. What is a merge conflict?
10. Why is it important to have a good README file?

7.2 Feedback Collection

Importance of Feedback

Collect feedback to improve future workshops!

Suggested Feedback Questions:

1. How would you rate the workshop overall? (1-5)
2. Was the pace of the workshop appropriate?
3. Which part did you find most useful?
4. Which part was most confusing?
5. Did the hands-on exercises help your understanding?
6. Do you feel confident using Git/GitHub now?
7. What topics would you like covered in more detail?
8. Any suggestions for improvement?

8 Troubleshooting Guide

Common Issues & Quick Fixes

Installation Problems

Issue: Git not recognized in terminal

Solution:

- Windows: Add Git to PATH environment variable
- Mac: Install Xcode Command Line Tools
- Linux: Use package manager (`sudo apt install git`)

Authentication Issues

Issue: GitHub authentication failed

Solution:

- Use Personal Access Token instead of password
- Generate token: GitHub Settings → Developer settings → Tokens
- Use SSH keys for easier authentication

Push/Pull Problems

Issue: "Repository not found"

Solution:

- Check repository URL is correct
- Verify you have access permissions
- Ensure repository exists on GitHub

Issue: "Failed to push some refs"

Solution:

- Run `git pull` first to get latest changes
- Resolve any merge conflicts
- Then `git push`

Merge Conflicts

Issue: Confused about conflict markers

Solution:

- Open file in text editor
- Look for `<<<<<<, =====, >>>>>>`
- Choose which code to keep
- Delete all conflict markers
- Save, stage, and commit

Appendix: Sample Repository Structure

```
my-project/
  .git/                # Git repository data (hidden)
  .gitignore           # Files to ignore
  README.md            # Project documentation
  index.html           # Main file
  style.css            # Stylesheet
  script.js            # JavaScript
  assets/              # Images, fonts, etc.
    images/
    fonts/
```

Sample .gitignore File

```
# Operating System Files
.DS_Store
Thumbs.db

# Editor Files
.vscode/
.idea/
*.swp

# Dependencies
node_modules/
venv/

# Build Files
dist/
build/
*.log

# Sensitive Information
.env
config.json
secrets.txt
```

End of Workshop

Thank you for attending!

Keep practicing and happy coding!



Prepared by **Muhammad Idrees**, Head of Workshops
FAST Data Science Society